MarketPulse – Milestone 3: Performance Evidence

Z2004: Database Management Systems | IIT Madras Zanzibar

Nidheesh Deepu Nair (ZDA24B013) · Anuj Gautam (ZDA24B033) · Shreyash Kumar Pandey (ZDA24B004) | Track A: RAG Pipeline | June 2026

1. Project Summary

MarketPulse is a Retrieval-Augmented Generation (RAG) pipeline that answers "why did this stock move?" by connecting structured market data (daily OHLCV prices, analyst ratings, macroeconomic indicators) with financial news articles. The database holds 16,063 rows across seven normalised tables (3NF) for 25 publicly traded companies. PostgreSQL was chosen over SQL Server because the Z2004 Track A guidelines require vector storage via pgvector or Pinecone; our schema uses a VECTOR(768) column with an HNSW index for cosine-similarity search, which is a pgvector-only feature. Keeping vectors and relational data in one engine simplifies both queries and deployment.

2. Objective and Methodology

This milestone evaluates query performance and demonstrates that targeted indexing produces measurable improvements. We identify three queries with inefficient access patterns, design B-Tree and HNSW indexes, present before/after EXPLAIN ANALYZE evidence, and implement a stored procedure and a trigger.

Table 1. Test environment.

Parameter	Value
Database	PostgreSQL 16.14 with pgvector 0.6.0
Machine	Intel Xeon @ 2.80 GHz, 1 core, 4 GB RAM, Ubuntu 24.04
Dataset	16,063 rows across 7 tables (stock_prices: 12,600 rows)
Method	EXPLAIN (ANALYZE, BUFFERS); indexes dropped for baseline, ANALYZE after each change

3. Results: Before and After Indexing

Query 1 – Date-range stock price lookup. Retrieves the 20 highest-volume trading days in October 2025 – the most common RAG pattern.

```
SELECT c.ticker, sp.trade_date, sp.close_price, sp.volume
FROM stock_prices sp JOIN companies c ON c.company_id = sp.company_id
WHERE sp.trade_date BETWEEN '2025-10-01' AND '2025-10-31'
ORDER BY sp.volume DESC LIMIT 20;
```

Before (no index):

```
Seq Scan on stock_prices sp (actual time=0.034..0.862 rows=575)
  Filter: trade_date BETWEEN '2025-10-01' AND '2025-10-31'
  Rows Removed by Filter: 12,025 | Buffers: shared hit=130
Execution Time: 1.760 ms
```

After (`idx_stock_prices_date`):

```
Bitmap Heap Scan on stock_prices sp (actual time=0.045..0.164 rows=575)
  Heap Blocks: exact=30
  -> Bitmap Index Scan on idx_stock_prices_date (Buffers: shared read=2)
Execution Time: 0.526 ms
```

Query 2 – Correlated subquery (bullish signal detection). Finds companies with $\geq$2 analyst "Buy" ratings AND positive average news sentiment. SubPlan 3 joins article_company with news_articles for each company. **Before** – SubPlan 3 (bottleneck):

```
Seq Scan on article_company ac (actual time=0.043..0.078 rows=60 loops=14)
  Filter: company_id = c.company_id
  Rows Removed by Filter: 1,440 per loop | Buffers: shared hit=98
Total Buffers: 1,212 | Execution Time: 5.575 ms
```

After (`idx_article_company_company`):

```
Index Scan using idx_article_company_company on article_company ac
  (actual time=0.004..0.012 rows=60 loops=14)
  Index Cond: company_id = c.company_id | Buffers: shared hit=41, read=3
Total Buffers: 1,126 | Execution Time: 4.151 ms
```

Query 3 – RAG pipeline date-range retrieval. Joins stock_prices, companies, article_company, and news_articles to pair prices with contemporaneous news. **Before:**

```
Seq Scan on news_articles na (actual time=0.011..0.366 rows=62)
  Filter: published_at >= '2024-06-01' AND < '2024-07-01'
  Rows Removed by Filter: 1,438 | Buffers: shared hit=72
Total Buffers: 162 | Execution Time: 2.343 ms
```

After (`idx_news_published_at`):

```
Bitmap Heap Scan on news_articles na (actual time=0.034..0.163 rows=62)
  Heap Blocks: exact=40
  -> Bitmap Index Scan on idx_news_published_at (Buffers: shared hit=2, read=1)
Total Buffers: 133 | Execution Time: 1.951 ms
```

Table 2. Performance summary.

Query	Before	After	Speedup	Buffer Reduction
Q1: Date-range prices	1.760 ms	0.526 ms	3.3×	134 → 36 (73%)
Q2: Correlated subquery	5.575 ms	4.151 ms	1.3×	1,212 → 1,126 (7%)
Q3: RAG retrieval	2.343 ms	1.951 ms	1.2×	162 → 133 (18%)

4. Interpretation of Execution Plans

Query 1 showed the largest gain. Without the B-Tree index on `trade_date`, the planner Sequential-Scanned all 12,600 rows, reading 130 buffer pages and discarding 12,025 non-matching rows. After creating `idx_stock_prices_date`, it switched to a Bitmap Index Scan: the index identified 575 matching locations in just 2 buffer reads, then fetched only 30 data pages. Execution time dropped 3.3× and buffers fell 73%. The DESC order benefits the common case of querying recent data first. **Query 2.** SubPlan 3 ran a Seq Scan on `article_company` for each of 14 qualifying companies, reading all 1,500 rows and discarding ∼1,440 per loop. The composite PK (`article_id`, `company_id`) has `article_id` as the leading column, so it cannot serve lookups by `company_id` alone. After creating `idx_article_company_company`, the planner replaced the Seq Scan with an Index Scan directly locating ∼60 rows per company. The overall 1.3× gain is modest because `news_articles` still scans fully in the same subplan, but the repeated 1,500-row filter loop is eliminated, a cost that scales linearly with table size. **Query 3.** The Seq Scan on `news_articles` (72 pages, 1,438 rows discarded) became a Bitmap Index Scan reading just 3 pages. The planner also introduced a Memoize node on `stock_prices`, caching results for repeated company IDs (38 hits vs 24 misses), further reducing redundant index probes. **A note on scale.** Absolute times are small because the dataset is ∼16,000 rows. In production with hundreds of thousands of records, Seq Scan costs grow linearly ($O(n)$) while indexed plans stay logarithmic ($O(\log n)$). The planner may also correctly choose a Seq Scan when the

filter is not selective enough (e.g., matching 70% of the table); this is optimal behaviour, not an index failure.

5. Index Design

Table 3. Complete index inventory.

Index Name	Type	Column(s)	Rationale
idx_stock_prices_date	B-Tree	trade_date DESC	Date-range filters are the #1 query pattern; DESC for recent-first.
idx_stock_prices_company	B-Tree	company_id	Per-company lookups when no date condition is present.
idx_news_published_at	B-Tree	published_at DESC	RAG retrieval filters news by publication time window.
idx_article_company_company	B-Tree	company_id	PK leads with article_id; this enables company_id lookups.
idx_analyst_ratings_company	B-Tree	(company_id, rating_date DESC)	Composite: equality on company + date-ordered access.
idx_news_embedding_hnsw	HNSW	embedding vector_cosine_ops	ANN search for RAG semantic retrieval (pgvector).

Index DDL. The exact `CREATE INDEX` statements used (also in `performance.sql`, Section 4):

```
CREATE INDEX idx_stock_prices_date
    ON stock_prices (trade_date DESC);

CREATE INDEX idx_stock_prices_company
    ON stock_prices (company_id);

CREATE INDEX idx_news_published_at
    ON news_articles (published_at DESC);

CREATE INDEX idx_article_company_company
    ON article_company (company_id);

CREATE INDEX idx_analyst_ratings_company
    ON analyst_ratings (company_id, rating_date DESC);

CREATE INDEX idx_news_embedding_hnsw
    ON news_articles
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);
```

Trade-offs. Each B-Tree index adds ∼1–3% storage overhead and slows INSERT/UPDATE operations. For MarketPulse this is acceptable: data is write-once (batch-imported per ingestion cycle) and read-heavy (queried by the RAG pipeline). The HNSW index has a higher build cost ($O(n \log n)$) but enables sub-linear ANN queries that would otherwise require an $O(n)$ brute-force scan.

6. Stored Procedure and Trigger

Stored procedure: `refresh_sentiment_summary()`. After each batch of news articles is ingested, the RAG pipeline needs per-company sentiment statistics. Computing this on every query (joining `article_company` and `news_articles`, grouping by company, averaging scores) is expensive. This function materialises the aggregates into a `company_sentiment_summary` table: one row per company with total articles, average sentiment, latest article date, and positive/negative/neutral counts (± 0.05 threshold). It returns the refreshed rows for verification. To run: `SELECT * FROM refresh_sentiment_summary();`

```
CREATE OR REPLACE FUNCTION refresh_sentiment_summary()
RETURNS SETOF company_sentiment_summary LANGUAGE plpgsql AS $$
BEGIN
  TRUNCATE company_sentiment_summary;
  INSERT INTO company_sentiment_summary (...)
    SELECT c.company_id, c.ticker, COUNT(na.article_id),
           ROUND(AVG(na.sentiment_score), 3),
           COUNT(*) FILTER (WHERE sentiment_score > 0.05), ...
    FROM companies c LEFT JOIN ... GROUP BY c.company_id, c.ticker;
  RETURN QUERY SELECT * FROM company_sentiment_summary;
END; $$;
```

Trigger: `trg_high_volatility` on `stock_prices`. A BEFORE INSERT trigger detects trading days where the intra-day range ((`high` − `low`) / `open`) exceeds 5%. The trigger function `fn_flag_high_volatility()` inserts an alert into `volatility_alerts` with the computed range percentage. This creates a pre-filtered queue of "days that need explaining" for the RAG pipeline, avoiding a full scan of 12,600 price rows. A test with open=100, high=110, low=95 (15% range) correctly produced an alert. To test:

```
INSERT INTO stock_prices (company_id, trade_date, open_price,
    high_price, low_price, close_price, volume)
VALUES (1, '2026-01-15', 100, 110, 95, 108, 5000000);
SELECT * FROM volatility_alerts;

CREATE OR REPLACE FUNCTION fn_flag_high_volatility()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_range_pct NUMERIC(6,2);
BEGIN
  v_range_pct := ROUND(100.0 * (NEW.high_price - NEW.low_price)
                       / NEW.open_price, 2);
  IF v_range_pct > 5.0 THEN
    INSERT INTO volatility_alerts (...) VALUES (...);
  END IF;
  RETURN NEW;
END; $$;
CREATE TRIGGER trg_high_volatility BEFORE INSERT ON stock_prices
FOR EACH ROW EXECUTE FUNCTION fn_flag_high_volatility();
```

7. Conclusions

Targeted indexing produced measurable improvements across all three benchmarked queries. Query 1 benefited most: `idx_stock_prices_date` reduced buffer accesses by 73% and tripled throughput by converting a Sequential Scan on 12,600 rows into a Bitmap Index Scan on 575 rows. Query 2 demonstrated that a simple B-Tree index on a junction table's non-leading column eliminates repeated full-table scans in correlated subqueries, a pattern whose cost scales multiplicatively with both table size and outer-loop iterations. Query 3 confirmed that `idx_news_published_at` targets the RAG pipeline's core access pattern, reducing the news scan from 72 buffer pages to 3. The stored procedure and trigger complement the indexing work. The procedure pre-computes sentiment aggregates (avoiding repeated expensive joins), and the trigger automatically flags volatile trading days (creating a lightweight alert table the pipeline queries directly). Together, indexed access paths, materialised aggregates, and event-driven alerting form a coherent performance strategy that will scale as more tickers, articles, and trading days are ingested.